

Universitatea Petrol și Gaze Ploiești
Facultatea de Litere și Științe
Specializarea Informatica, An III

PROIECT ISW

- Aplicatie Java -

Manager de Proiect: Branzea Ana-Maria
Programator: Cosma Tania
Programator: Dinu Cosmin
Programator: Banica Adelin
Programator: Cobzaru Cosmin

Cuprins

1. Introducere.....	pg. 2
2. Management.....	pg. 2
3. Cerintele proiectului.....	pg. 5
4. Modelul sistemului.....	pg. 9
5. Soluții de proiectare, implementare si testare.....	pg. 14
6. Ghidul Utilizatorului.....	pg.32
7. Documentatie de sistem.....	pg. 43

1. Introducere

Ingineria Software reprezinta ramura informaticii care se ocupa cu proiectarea, dezvoltarea și menținerea sistemelor software, oferind soluții la diverse probleme în lumea digitală. Aceasta disciplina reunește creativitatea cu tehnicalitatea pentru a crea produse software eficiente, fiabile și adaptate la nevoile societății. Cu rădăcini ce se întind pe decenii, ingineria software a devenit un domeniu essential, alimentando progresul in diverse industries. De la aplicații mobile pana la sisteme complexe de operare care austin infrastructura globala.

Cu intenția de a ne implica în acest domeniu, am creat o echipa de programatori creativi și capabili pentru a dezvolta o aplicatie care rezolva probleme ce implica structuri de date.

Obiectivele noastre sunt de a livra o aplicație completă, usor de utilizat cu o interfata grafica intuitiva, care sa ofere rezolvări pentru problemele propuse cu structuri de date.

Pentru a ne atinge obiectivele propuse, vom adopta modelul Waterfall pentru planificarea etapelor de proiectare și paradigma Agile pentru abordarea provocărilor în contextul livrării aplicației. Proiectul este împărțit în patru etape: Planificare Proiect, Rezolvare Probleme și Testare, Proiectare Aplicație și Testare, Documentatie.

Echipa este formată din cinci persoane; Un manager de proiect și patru programatori, iar planificarea și organizarea se realizeaza prin colaborarea întregului personal, avand la baza dezbaterea de pareri, opinii, propuneri si idei. Astfel, prin comunicare și colaborare vom duce la bun sfârșit proiectul.

2. Management

Pentru a asigura livrarea produsului într-un interval de timp bine definit, am folosit un orar ce are ca functionalitati: impartirea proiectului pe repere manageriale, monitorizarea și managerierea sarcinilor membrilor echipei, monitorizarea și managerierea termenilor limita atat pentru membrii echipei cat si pentru livrarea produsului și menținerea evidenței rolurilor personalului implicat. Orarul poate fi accesat de către fiecare membru al echipei, avand posibilitatea și libertatea de a modifica și adaugă înregistrări în scopul livrării produsului.

Procesul de livrare este împărțit în patru repere manageriale, fiecare milestone fiind vital în succesul aplicației software:

- Planificare Proiect
- Rezolvare Probleme + Testare
- Proiectare Aplicație + Testare
- Documentatie

1. Planificare Proiect

Etapa de planificare a proiectului este stagiul primar al proiectului, în care toți participanții au luat parte la multiple discuții și întâlniri în care s-au dezbătut multiple idei, propuneri și opinii cu scopul realizării proiectului. În această etapă s-au luat cele mai multe decizii referitoare la demersul proiectului pe toată perioada de livrare, încercând prin cooperare să ajungem la un plan solid pe care să-l urmăm până la finalizarea proiectului. S-a utilizat serviciul Discord pentru a facilita schimbul de informații prin intermediul canalelor de comunicare text: general, doc, cerințe proiect, pentru doc și canalelor de comunicare audio: lounge, sala de studiu 1, sala de studiu 2.

2. Rezolvare Probleme + Testare

Etapa de rezolvare de probleme și testare este cea în care am început să lucrăm în adevăratul sens al cuvântului la aplicații. Problemele au fost repartizate fiecărui participant spre rezolvare. Atât prin munca în echipă cât și individuală am reușit să rezolvăm problemele în timp util pentru a progresa spre următoarea etapă. După ce o problemă era rezolvată, aceasta era testată pentru a asigura progresul proiectului fără a deroga de la planul inițial.

3. Proiectare Aplicație + Testare

Etapa de proiectare a aplicației și testare este stagiul în care problemele rezolvate anterior sunt implementate într-o singură aplicație completă cu interfața grafică intuitivă, ușor de navigat. Tot în această etapă au fost realizate teste pe aplicație pentru a asigura funcționalitatea completă și au fost efectuate retusuri pentru a îmbunătăți aspectul vizual al acesteia.

4. Documentație

Etapa de documentație este cea în care a fost scrisă documentația sistemului. Au avut loc discuții și dezbateri referitoare la schițarea și formatul acesteia înainte de a o livra. Tot în această etapă au fost scrise manualele de sistem și de utilizator, iar spre finalul termenului de livrare au avut loc revizuri asupra acestora pentru a asigura calitatea proiectului.

Sarcini, roluri și resurse necesare

Sarcinile majore au fost repartizate în funcție de specialitatea fiecărui membru al echipei. Problemele au fost rezolvate de fiecare membru participant la proiect. Implementarea rezolvărilor într-o aplicație software ce dispune de o interfață grafică a fost atribuită managerului de echipă. Testele pentru asigurarea funcționalității produsului au fost efectuate de către fiecare membru al echipei. Referitor la organizare, toată echipa a participat activ la proiect, motiv pentru care organizarea a fost omniprezentă pe perioada tuturor reperelor manageriale. Resursele

necesare finalizarii proiectului au fost în mare masura de timp, motiv pentru care datele de start si finalizare au fost foarte bine definite.

Orarul utilizat pentru manageriere

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	AA	AB	AC	AD	AE	AF	AG	AH	AI	AJ	AK	
1	TITLU PROIECT			PROIECT ISW																																		
2	PARTICIPANTI PROIECT			Branzea Ana-Maria - Manager de Proiect Dinu Cosmin - Programator Cosma Tania - Programator Banica Adelin - Programator Cobzaru Cosmin - Programator		DATE		1/10/24																														
3																																						
4								DECEMBRIE															IANUARIE															
5	TASK TITLE			TASK OWNER	START DATE	DEADLINE	18	19	20	21	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
6																																						
7	1	Planificare Project																																				
8			Discutie echipa	@everyone	18.12.2023																																	
9			Planificare Organizare	@everyone	19.12.2023																																	
10			Discutie echipa	@everyone	21.12.2023																																	
11			Planificare Implementare si Proiectare	@everyone	26.12.2023																																	
12			Planificare Testare	@everyone	28.12.2023																																	
13			Planificare Design	@everyone	04.01.2024																																	
14			Planificare Documentatie	@everyone	05.01.2024																																	
15	2	Rezolvare Probleme + Testare																																				
16			Problema 53	Branzea Ana-Maria	18.12.2023	3.01.2024																																
17			Problema 39	Cobzaru Cosmin	18.12.2023	3.01.2024																																
18			Problema 48	Cosma Tania	18.12.2023	3.01.2025																																
19			Problema 7	Dinu Cosmin	18.12.2023	3.01.2026																																
20			Problema 5	Banica Adelin	18.12.2023	3.01.2027																																
21			Testare Rezolvari	@everyone	18.12.2023	10.01.2024																																
22																																						
23	3	Proiectare Aplicatie + Testare																																				
24			Interfata problema 39	Branzea Ana-Maria	18.12.2023	03.01.2024																																
25			Interfata problema 53	Branzea Ana-Maria	18.12.2023	06.01.2024																																
26			Interfata problema 7	Branzea Ana-Maria	18.12.2023	07.01.2024																																
27			Interfata problema 5	Branzea Ana-Maria	18.12.2023	08.01.2024																																
27			Interfata problema 5	Branzea Ana-Maria	18.12.2023	08.01.2024																																
28			Testare Implementare	Branzea Ana-Maria	27.12.2023	09.01.2024																																
29			Retusuri finale	Branzea Ana-Maria	09.12.2023	10.01.2024																																
30			Testare Software	@everyone	18.12.2023	11.01.2024																																
31	4	Documentatie																																				
32			Discutie Draft Documentatie	@everyone	05.01.2023	08.01.2023																																
33			Schitare Documentatie	@everyone	10.01.2023	13.01.2023																																
34			Manual Sistem	@everyone	14.01.2023	15.01.2023																																
35			Manual Utilizator	@everyone	15.01.2023	16.01.2023																																
36			Formatare Documentatie	@everyone	16.01.2023	17.01.2023																																
37			Revizuire Documentatie	@everyone	17.01.2023	18.01.2023																																

3. Cerințele proiectului

Definirea Informala a cerințelor

Trebuie realizata o aplicație software conform următoarelor cerințe:

- Interfata grafica intuitiva, astfel incat fiecare meniu al aplicației poate fi navigat apasand maxim trei butoane.
- Meniu principal prin intermediul căruia se pot accesa toate ferestrele funcționale ale aplicației.
- Butoane de navigare.
- Enunțurile problemelor trebuiesc afișate în ferestrele de rezolvare pentru context
- Ferestrele de rezolvare trebuie sa contina zone de text în care utilizatorii vor introduce valori.
- Ferestrele de rezolvare trebuie sa contina butoane prin intermediul cărora se vor efectua operațiile problemelor.
- Ferestrele de rezolvare trebuie sa contina campuri de text în care rezultatul va fi afișat.

Specificarea formala a cerintelor si a sistemului software

1. Cerinte de interfata grafica

1.1 Interfata Grafica Intuitiva:

- Aplicatia trebuie sa dispuna de o interfata grafica intuitiva, astfel incat fiecare meniu al aplicației sa poată fi navigat prin apasarea a maxim trei butoane.
- Utilizatorii vor avea acces la functionalitatile aplicației prin intermediul butoanelor.
- Navigarea in meniuri si utilizarea functiilor principale se va face in mod intuitiv.

1.2 Meniu Principal:

- Aplicația trebuie sa includă un meniu principal prin intermediul căruia se pot accesa toate ferestrele funcționale ale aplicației.
- Utilizatorii pot accesa oricare fereastra folosind meniul principal.
- Pseudocod: Clasa Meniu extinde JFrame

Metoda Meniu()

apelează initComponents()

Metoda initComponents()

Inițializează jPanel1, jTextField1, jButton3, jButton2, jButton4, jButton1, jButton5, jLabel1

Setează proprietățile pentru fiecare componentă (de exemplu, dimensiunea, culoarea, fontul)

Adaugă acțiuni pentru fiecare buton (jButton1, jButton2, jButton3, jButton4, jButton5)

Adaugă toate componentele în jPanel1

Adaugă jPanel1 în fereastra JFrame

Metoda jButton2ActionPerformed(ActionEvent evt)

Închide fereastra curentă

Deschide fereastra CosminDFrame

Metoda jButton1ActionPerformed(ActionEvent evt)

Închide fereastra curentă

Deschide fereastra interfatapb53

Metoda jButton3ActionPerformed(ActionEvent evt)

Închide fereastra curentă

Deschide fereastra CosminCFrame

Metoda jButton5ActionPerformed(ActionEvent evt)

Închide fereastra curentă

Deschide fereastra AdelinFrame

Metoda jButton4ActionPerformed(ActionEvent evt)

Închide fereastra curentă

Deschide fereastra Tania

Metoda main(String args[])

Rulează nouă instanță a clasei Meniu

1.3 Butoane de Navigare:

- Fiecare fereastră funcțională trebuie să conțină butoane de navigare.
- Butoanele de navigare vor facilita accesul eficient la funcționalitatea specifică fiecărei ferestre.
- Numărul maxim de butoane necesare pentru navigarea în interiorul ferestrei este limitat la trei.
- Pseudocod: Clasa Meniu extinde JFrame

Funcția initComponents()

Inițializează jPanel1, jTextField1, jButton1, jButton2, jButton3, jButton4, jButton5, jLabel1

Setează proprietățile pentru fiecare componentă (dimensiune, culoare, text, etc.)

Setează ascultătorii de acțiuni pentru fiecare buton

Sfârșit Funcție

Când se face clic pe jButton1

Închide fereastra curentă

Creează o nouă instanță de Interfatapb53

Face noua fereastră vizibilă

Sfârșit Când

Când se face clic pe jButton2

Închide fereastra curentă

Creează o nouă instanță de CosminDFrame

Face noua fereastră vizibilă

Sfârșit Când

Când se face clic pe jButton3

Închide fereastra curentă

Creează o nouă instanță de CosminCFrame

Face noua fereastră vizibilă

Sfârșit Când

Când se face clic pe jButton4

Închide fereastra curentă

Creează o nouă instanță de Tania

Face noua fereastră vizibilă

Sfârșit Când

Când se face clic pe jButton5

Închide fereastra curentă
Creează o nouă instanță de AdelinFrame
Face noua fereastră vizibilă
Sfârșit Când

Funcția main()
Setează aspectul și senzația UI
Creează și afișează o nouă fereastră Meniu
Sfârșit Funcție

Sfârșit Clasă

2. Cerințe pentru Ferestre de Rezolvare

2.1 Enunțuri Probleme:

- Ferestrele de rezolvare trebuie să afișeze enunțurile problemelor pentru context.

2.2 Zone de Text pentru introducerea Valori:

- Ferestrele de rezolvare trebuie să conțină zone de text în care utilizatorii vor introduce valorile necesare.
- Zonele de text trebuie să permită introducerea valorilor necesare pentru rezolvarea problemei.
- Validările necesare vor fi aplicate pentru asigurarea introducerii datelor corecte.

2.3 Butoane pentru Efectuarea Operațiilor:

- Ferestrele de rezolvare trebuie să conțină butoane prin intermediul cărora se vor efectua operațiile problemelor.
- Fiecare buton trebuie să fie etichetat clar cu operația pe care o va efectua.
- Operațiile vor fi realizate doar după ce utilizatorul apasă explicit butonul asociat.

2.4 Campuri de Text pentru Afișarea Rezultatelor:

- Ferestrele de rezolvare trebuie să conțină campuri de text în care rezultatul va fi afișat.

- Rezultatele obținute în urma operațiilor vor fi afișate în mod clar în campurile de text.
- Formatul afișării rezultatelor va fi coerent și ușor de înțeles.

4. Modelul sistemului

În cadrul acestei secțiuni explorăm și detaliem modelul sistemului pentru cinci probleme distincte. Fiecare problemă aduce în prim-plan cerințe specifice, iar abordarea lor eficientă implică implementarea unui sistem coerent și robust. Vom explora modul în care structurile de date și algoritmi aleși sunt utilizați pentru a rezolva fiecare problemă în parte.

Problemele abordate includ calculul reuniunii și intersecției a două mulțimi reprezentate de o stivă și o coadă, interclasarea a două liste ordonate crescător, eliminarea cheilor pozitive dintr-o listă simplu înălțuită, construirea unei matrice asociate unei cozi complexe și afișarea elementelor negative dintr-o listă dublu înălțuită. Fiecare dintre aceste probleme evidențiază aspecte cheie ale ingineriei software, incluzând manipularea eficientă a datelor, organizarea adecvată a informațiilor și implementarea algoritmilor optimați.

Prin urmare, ne propunem să oferim o prezentare comprehensivă a modului în care fiecare problemă este abordată, evidențiind alegerile de proiectare, structurile de date implicate și procesele de control al datelor utilizate pentru a asigura o rezolvare eficientă și corectă a fiecărei provocări în parte.

Problema 53. Fie două mulțimi oarecare, una reprezentată cu o stivă și una cu o coadă. Se cere să calculeze reuniunea și intersecția lor și să se reprezinte, cu o stivă, respectiv coadă.

Date de intrare:

- Coadă: O structură de date care respectă principiul "primul intrat, primul ieșit" (First-In-First-Out, FIFO)
- Stivă: O structură de date care respectă principiul "ultimul intrat, primul ieșit" (Last-In-First-Out, LIFO)

Mod de citire a datelor de intrare:

Utilizatorul furnizează aceste date prin intermediul interfeței grafice, introducând numere de tip long separate prin spațiu pentru coadă și stivă. După introducerea datelor, utilizatorul apasă butonul "Adaugă" pentru a le include în structurile de coadă și stivă.

Date de ieșire:

- Coadă rezultat: O coadă care conține elementele rezultate în urma operației de intersecție între stivă și coadă inițiale introduse de utilizator.

- Stivă rezultat: O stivă care conține elementele rezultate în urma operației de reuniune dintre stiva și coada inițiale introduse de utilizator.

Mod de procesare și de control al datelor:

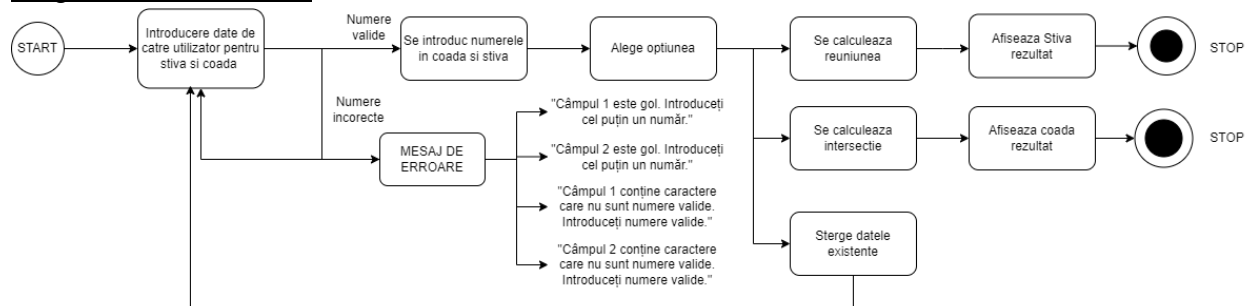
Modul de procesare și control implică formarea stivei și a coadei prin intermediul funcțiilor specifice precum push și insert. Rezultatele sunt generate cu ajutorul algoritmilor specifici pentru operațiile de reuniune și intersecție.

Pentru a asigura integritatea datelor, se efectuează verificări și se afișează mesaje de eroare în cazul în care:

- Campurile nu sunt completate corect (adică, se încearcă adăugarea de numere cu virgulă).
- Sunt adăugate caractere nepermise.
- Unul dintre câmpuri este lăsat gol.

Aceste măsuri asigură că datele introduse sunt valide și pot fi procesate corespunzător, prevenind astfel erori sau rezultate incorecte.

Diagrama de activitate:



Problema 39. Să se interclaseze 2 liste ordonate crescător (în prealabil) – interclasarea presupune construcția unei liste ordonate crescător, treptat, din cele 2 liste ordonate crescător, astfel: la un moment dat se compară 2 elemente, unul din prima listă și unul din a doua listă și cel mai mic dintre ele va fi “propagat” în lista rezultat, după care procesul se reia. Ex. I1: 1 3 5 20 40 I2: 2 4 5 15 25 I: 1 2 3 4 5 5 15 20 25 40

Date de intrare:

- Valori numerice reale de tip double (ex: 3.14, 42, -15.8)

Mod de citire a datelor de intrare:

- Datele de intrare sunt introduse de către utilizator prin intermediul interfeței grafice. Valorile introduse sunt delimitate prin spații.

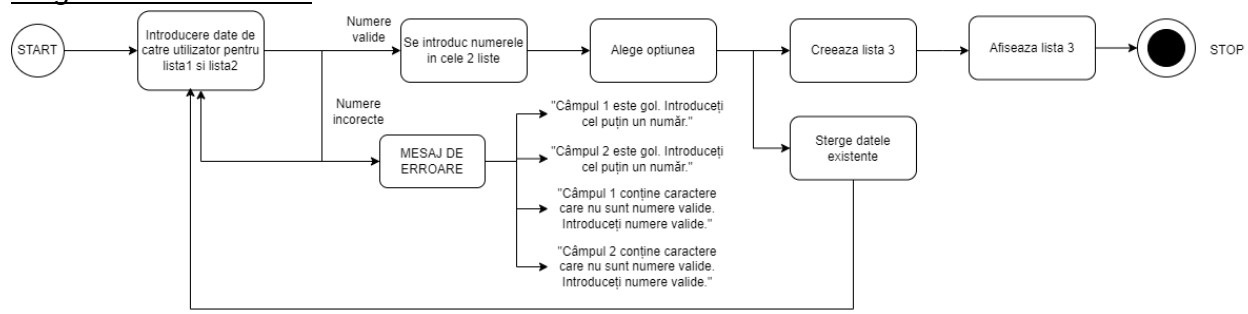
Date de iesire:

- O lista de noduri rezultată în urma procesării datelor. Lista contine nodurile (valorile) ordonate crescator, delimitate printr-un spațiu pentru lizibilitate.

Mod de procesare și de control al datelor:

- Datele sunt introduse în doua structuri de date de tip lista simplu inlantuita pentru a fi manipulate ulterior.
- Functia Sort() efectuează sortarea listei folosind algoritmul Bubble Sort.
- Functia Insert() insereaza un nod în ordine crescatoare in lista rezultate.
- Datele sunt validate înaintea efectuării operațiilor, adică se efectuează verificarea dacă lista este boala sau dacă valoarea trebuie inserată la început sau pe o poziție specifică, dacă sunt introduse valori diferite de tipul de date double.

Diagrama de activitate:



Problema 5. Să se afișeze elementele negative aflate în intervalul $[a,b]$ dintr-o listă dublu înlănțuită

Date de intrare: Un număr întreg care va fi adăugat la lista dublu înlănțuită. Limita inferioară $[a]$ a intervalului în care se caută numerele negative. Limita superioară $[b]$ a intervalului în care se caută numerele negative.

Mod de citire a datelor de intrare: Utilizatorul furnizează aceste date prin intermediul interfeței grafice, introducând numerele de tip int în câmpurile de text corespunzătoare. După introducerea datelor, utilizatorul poate apăsa butonul “Adauga Element” pentru a adăuga elementul la listă sau butonul “Afiseaza Negative” pentru a afișa numerele negative din intervalul specificat.

Date de ieșire: O listă dublu înlănțuită care conține elementele adăugate de utilizator. Un șir de caractere care reprezintă numerele negative din intervalul $[a, b]$ din listă.

Mod de procesare și de control al datelor: Modul de procesare și control implică adăugarea elementelor la lista dublu înlănțuită prin intermediul metodei adaugaLaSfarsit și afișarea numerelor negative din intervalul specificat prin intermediul metodei afiseazaNegative. Pentru a asigura integritatea datelor, se efectuează verificări și se afișează mesaje de eroare în cazul în care:

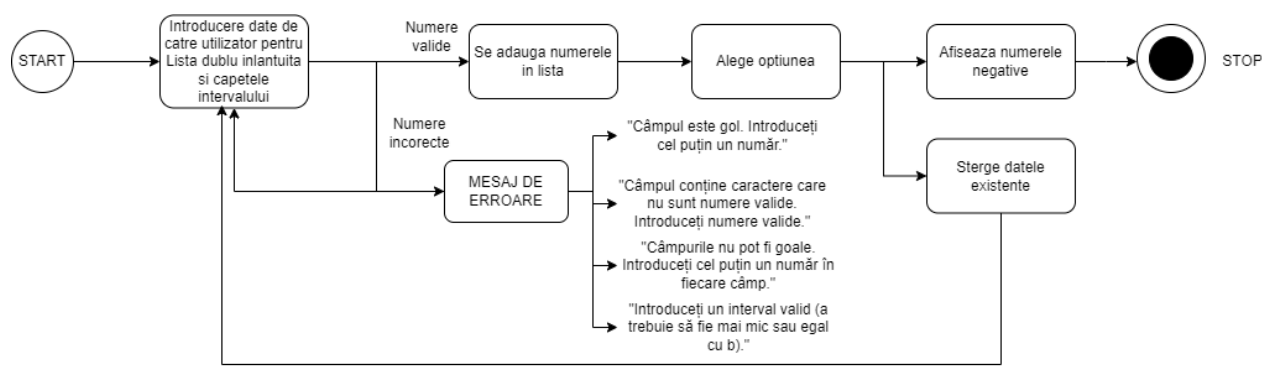
Campurile **nu** sunt completate corect (adică, se încearcă adăugarea de numere cu virgulă).

Sunt adăugate caractere nepermise.

Unul dintre câmpuri este lăsat gol.

Aceste măsuri asigură că datele introduse sunt valide și pot fi procesate corespunzător, prevenind astfel erori sau rezultate incorecte.

Diagrama de activitate:



Problema 7. Eliminarea dintr-o listă simplu înlănțuită a cheilor pozitive.

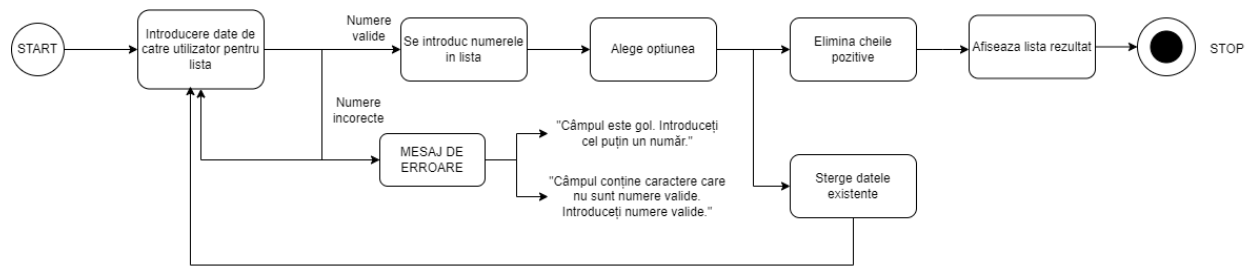
Date de intrare: O lista de numere de tip double pentru lista dublu înlănțuită.

Mod de citire a datelor de intrare: Utilizatorul furnizează aceste date prin intermediul metodei `tinsertFirst(double dd)`, care adaugă un element la începutul sau la sfârșitul listei, respectiv.

Date de ieșire: O listă dublu înlănțuită care conține elementele rămase după ștergerea elementelor pozitive.

Mod de procesare și de control al datelor: Modul de procesare și control implică adăugarea elementelor la lista dublu înlănțuită prin intermediul metodelor `insertFirst(double dd)` și ștergerea elementelor pozitive din lista intermediul metodei `stergeNodPoz(int info)`. Lista poate fi afișată folosind metoda `afisare()`.

Diagrama de activitate:



Problema 48. Se dă o coadă de forma: $l: a_{11} a_{12} a_{13} a_{14} .. a_{1n} \rightarrow a_{21} a_{22} a_{23} a_{23} .. a_{2n} \rightarrow ... \rightarrow a_{m1} a_{m2} a_{m3} .. a_{mn} /$, cu m și n date. Se cere să se construiască matricea asociată care conține m linii și n coloane, pe fiecare linie fiind elementele din câmpurile informație ale uneia din celulele listei, astfel prima linie va conține $a_{11} a_{12} a_{13} a_{14} .. a_{1n}$, a doua $a_{21} a_{22} a_{23} a_{24} .. a_{2n}$ samd.

Date de intrare: Coada: O structură de date care respectă principiul “primul intrat, primul ieșit” (First-In-First-Out, FIFO). Coada este reprezentată ca un șir de caractere în care elementele sunt separate prin spații și liniile sunt separate prin simbolul “→”.

Mod de citire a datelor de intrare: Utilizatorul furnizează aceste date prin intermediul interfeței grafice, introducând numerele de tip int separate prin spațiu pentru coadă. După introducerea datelor, utilizatorul apasă butonul “Procesează” pentru a iniția procesarea cozii.

Date de ieșire: O matrice care conține m linii și n coloane, pe fiecare linie fiind elementele din câmpurile informație ale uneia din celulele listei.

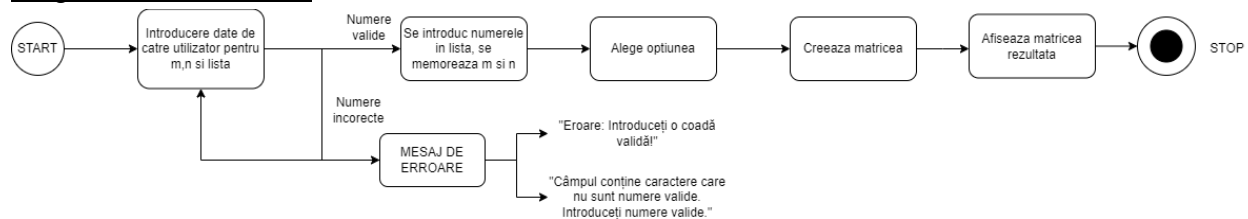
Mod de procesare și de control al datelor: Modul de procesare și control implică formarea cozii prin intermediul funcției insereaza. Rezultatele sunt generate cu ajutorul algoritmului specific pentru construirea matricei asociate. Pentru a asigura integritatea datelor, se efectuează verificări și se afișează mesaje de eroare în cazul în care:

Câmpurile nu sunt completate corect (adică, se încearcă adăugarea de numere cu virgulă).

Sunt adăugate caractere nepermise.

Unul dintre câmpuri este lăsat gol.

Diagrama de activitate:



5. Soluții de proiectare, implementare si testare

Problema 53. Fie două mulțimi oarecare, una reprezentată cu o stivă și una cu o coadă. Se cere să calculeze reuniunea și intersecția lor și să se reprezinte, cu o stivă, respectiv coadă.

Structuri de Date

LinkA Class

Clasa LinkA conține următoarele funcții:

- Constructor LinkA(long data): Inițializează un obiect LinkA cu un câmp de date specificat.
- displayLink(): Afișează valoarea câmpului de date al elementului.

LinkedListA Class

Clasa LinkedListA include următoarele funcții:

- Constructor LinkedListA(): Inițializează o listă simplu înlănțuită fără elemente.
- isEmpty(): Verifică dacă lista este goală.
- insertFirst(long dd): Inserează un nou element la începutul listei.
- displayList(): Afișează conținutul listei.
- clear(): Elimină toate elementele din listă, setând primul element la null.

FirstLastListA Class

Clasa FirstLastListA extinde LinkedListA și adaugă funcțiile:

- Constructor FirstLastListA(): Inițializează o listă cu primul și ultimul element nule.
- insertLast(long dd): Inserează un nou element la sfârșitul listei.

LinkQueueA Class

Clasa LinkQueueA utilizează FirstLastListA și include funcțiile:

- Constructor LinkQueueA(): Inițializează o coadă utilizând FirstLastListA.

- isEmpty(): Verifică dacă coada este goală.
- insert(long j): Inserează un nou element la sfârșitul cozii.
- displayQueue(): Afișează conținutul cozii.
- intersectie(LinkStackA stiva): Realizează operația de intersecție între coadă și stivă.
- displayAsString(): Returnează reprezentarea sub formă de șir a cozii.

LinkStackA Class

Clasa LinkStackA utilizează LinkListA și include funcțiile:

- Constructor LinkStackA(): Inițializează o stivă utilizând LinkListA.
- push(long j): Adaugă un nou element la începutul stivei.
- isEmpty(): Verifică dacă stiva este goală.
- displayStack(): Afișează conținutul stivei.
- reuniune(LinkQueueA coada): Realizează operația de reuniune între stivă și coadă.
- displayAsString(): Returnează reprezentarea sub formă de șir a stivei.

Operații Principale

Intersecție (Button 2):

```
LinkQueueA intersectie(LinkStackA stiva) {
    LinkQueueA rezultat=new LinkQueueA();
    HashSet<Long> set = new HashSet<>();

    // Adăugăm elementele din stivă în set
    LinkA currentStack = stiva.lista.first;
    while (currentStack != null) {
        set.add(currentStack.dData);
        currentStack = currentStack.next;
    }

    // Adăugăm în coadă doar elementele care sunt și în set
    LinkA current = lista.first;
    while (current != null) {
        if (set.contains(current.dData)) {
            rezultat.insert(current.dData);
        }
        current = current.next;
    }
    return rezultat;
}
```

- Pentru operația de intersecție, se utilizează metoda intersectie din clasa LinkQueueA. Această metodă primește o referință către o stivă (stiva) și creează o coadă nouă (rezultat) care va conține doar elementele comune între coadă și stivă.
- Metoda utilizează o structură de date HashSet pentru a eficientiza verificarea existenței elementelor. Elementele din stivă sunt adăugate în HashSet, iar apoi, parcurgând lista cozi, se adaugă în coada rezultat doar elementele care există și în HashSet. La final, coada rezultat este returnată.

Reuniune (Button 1):

```
LinkStackA reuniune(LinkQueueA coada) {
    LinkStackA rezultat = new LinkStackA();
    HashSet<Long> set = new HashSet<>();

    LinkA current = lista.first;
    while (current != null) {
        rezultat.push(current.dData);
        set.add(current.dData);
        current = current.next;
    }

    LinkA currentQueue = coada.lista.first;
    while (currentQueue != null) {
        if (!set.contains(currentQueue.dData)) {
            rezultat.push(currentQueue.dData);
            set.add(currentQueue.dData);
        }
        currentQueue = currentQueue.next;
    }
    return rezultat;
}
```

- Operația de reuniune între stivă și coadă este implementată în metoda reuniune din clasa LinkStackA. Această metodă primește o referință către o coadă (coada) și creează o stivă nouă (rezultat) care va conține toate elementele unice din ambele structuri.
- Metoda utilizează, similar cu intersecția, o structură de date HashSet pentru a eficientiza adăugarea și verificarea elementelor unice. Elementele din stivă sunt adăugate în HashSet și în stiva rezultat, iar apoi, parcurgând lista cozii, se adaugă în stivă doar elementele care nu există deja în HashSet. La final, stiva rezultat este returnată.

Ștergere Date (Button 4)

- Operația de ștergere a datelor este implementată în metoda clear din clasele LinkListA și FirstLastListA. Această metodă setează primul element al listei la null, eliminând astfel toate elementele din listă.

```
private void jButton4ActionPerformed(java.awt.event.ActionEvent evt) {
    stiva.lista.clear(); // Elimină toate elementele din stivă
    coada.lista.clear(); // Elimină toate elementele din coadă
    jTextField1.setText("");
    jTextField2.setText("");
    jTextArea2.setText("S-au sters datele din coada si stiva. Introduceți altele!");
}
```

Adăugare Date (Button 3):

- Operația de adăugare a datelor este realizată prin metoda insert din clasele LinkQueueA și LinkStackA. Această metodă primește datele din jTextField1

```
private void jButton3ActionPerformed(java.awt.event.ActionEvent evt) {  
    String textCoadă = jTextField1.getText();  
    String textStivă = jTextField2.getText();  
  
    // Adaugă numerele din jTextField1 în coadă (LinkQueueA)  
    String[] numereTextCoadă = textCoadă.split(" ");  
  
    for (String numarText : numereTextCoadă) {  
        try {  
            long numar = Long.parseLong(numarText);  
            coada.insert(numar);  
        } catch (NumberFormatException e) {  
            // cazul în care input-ul nu este un număr  
            JOptionPane.showMessageDialog(this, "Introduceți doar numere valide în câmpul pentru coadă.");  
            return;  
        }  
    }  
  
    // Adaugă numerele din jTextField2 în stivă (LinkStackA)  
    String[] numereTextStivă = textStivă.split(" ");  
  
    for (String numarText : numereTextStivă) {  
        try {  
            long numar = Long.parseLong(numarText);  
            stiva.push(numar);  
        } catch (NumberFormatException e) {  
            // cazul în care input-ul nu este un număr  
            JOptionPane.showMessageDialog(this, "Introduceți doar numere valide în câmpul pentru stivă.");  
            return;  
        }  
    }  
}
```

(coadă) și jTextField2 (stivă), le procesează și le adaugă în structurile de date respective.

- Datele sunt preluate sub formă de șir, separate și convertite în numere întregi (long). Aceste numere sunt apoi inserate în coadă sau stivă, în funcție de operație. Metoda se ocupă și de tratarea excepțiilor, afișând mesaje de eroare dacă input-ul nu este valid.

Testare

Testarea operațiilor este realizată în cadrul interfeței grafice

prin evenimente specifice fiecărui buton (Button 2, Button 1, Button 4, Button 3). Datele sunt introduse în jTextField1 și jTextField2, iar rezultatele sunt afișate în JTextArea2.

Pentru a ilustra cum lucrează aplicația, iată câteva exemple de date de intrare pentru fiecare operație:

Exemplu pentru Reuniune (Button 2):

Date de intrare:

- jTextField1 (coadă): 1 2 3
- jTextField2 (stivă): 4 5 6 7

Rezultat (Intersecție):

Reuniune:

Stivă (top-->bottom)- 3 2 1 4 5 6 7

Exemplu pentru intersecție(Button 1):

Date de intrare:

- jTextField1 (coadă): 1 2 3
- jTextField2 (stivă): 3 4 5 1

Rezultat (Reuniune):

Intersecție:

Coadă (front-->rear)- 1 3

Exemplu pentru Ștergere Date (Button 4):

Date de intrare:

- jTextField1 (coadă): 1 2 3
- jTextField2 (stivă): 3 4 5 1

Rezultat(Ștergere Date):

S-au șters datele din coadă și stivă. Introduceți altele!

Exemplu pentru Adăugare Date (Button 3) cu Excepții:

Date de intrare:

- jTextField1 (coadă): 10 20 abc
- jTextField2 (stivă): 30 40 50

Rezultat în JTextArea2 (Adăugare Date cu Excepție):

Mesaj de eroare:

Introduceți doar numere valide în câmpul pentru coadă.

Problema 7. Eliminarea dintr-o listă simplu înlănțuită a cheilor pozitive.

Structura Proiectului:

1.1 Clasa Link:

- Reprezintă un nod într-o listă simplu înlănțuită.
- Are un membru dData de tip double care stochează datele nodului.
- Conține un membru next de tip Link care indică către nodul următor.
- Metoda displayLink() afișează datele nodului.

1.2 Clasa LinkList:

- Reprezintă o listă simplu înlănțuită.

- Membrul first indică către primul nod din listă.
- Metoda displayList() afișează toate nodurile din listă.
- Metoda isEmpty() returnează true dacă lista este goală.
- Metoda insertFirst(double dd) inserează un nou nod la începutul listei.
- Metoda stergeNodPoz() elimină nodurile cu dData pozitiv.
- Metoda clear() golește lista.

Implementare Funcționalități:

2.1 Metoda jButton1ActionPerformed:

- Colectează datele introduse într-un câmp text (jTextField2).
- Verifică dacă câmpul este gol sau conține numere valide.
- Adaugă numerele valide în lista (lista) folosind metoda insertFirst.

2.2 Metoda jButton2ActionPerformed:

- Golește lista (lista), câmpul text (jTextField2), și afișează un mesaj de confirmare.

2.3 Metoda jButton3ActionPerformed:

- Afișează noua listă după eliminarea nodurilor cu dData pozitiv folosind metoda stergeNodPoz.
- Afișează un mesaj corespunzător dacă lista este goală.

2.4 Metoda jButton4ActionPerformed:

- Închide fereastra curentă și afișează un meniu nou (Meniu).

Testare:

Testarea Inserării de Date:

1. Inserare de Date Valide:

- jTextField2 (lista): 1 -2 4 7 8 -12
- Rezultat (jTextArea): Noua lista:
-12.0 -2.0

2. Camp gol

- jTextField2 (lista):
- Rezultat (jTextArea): Câmpul este gol. Introduceți cel puțin un număr.

3. Caractere invalide

- jTextField2 (lista): abc 1 2

- Rezultat (jTextArea): Câmpul conține caractere care nu sunt numere valide. Introduceți numere valide.

Testarea Ștergerii Datelor:

- jTextField2 (lista): 1 -2 4
- Rezultat (jTextArea): S-au sters datele din lista. Introduce-ti altele!

Problema 5. Să se afișeze elementele negative aflate în intervalul [a,b] dintr-o listă dublu înălțuită

Listele dublu înălțuite sunt structuri de date dinamice omogene. Ele au aceleași caracteristici de bază ca și listele simplu înălțuite. Diferența față de acestea constă în faptul că, pentru fiecare nod, se reține și adresa elementului anterior, ceea ce permite traversarea listei în ambele direcții.

Programul ListaDubluInlantuita implementează o structură de date de tip listă dublu înălțuită. Lista poate fi utilizată pentru a stoca și a manipula valori întregi.

Acest program implementează o structură de date de tip listă dublu înălțuită și oferă două operații:

*Adăugarea unui element la sfârșitul listei

*Afișarea elementelor negative dintr-un interval dat

Clase, variabile și metode:

1. Clasa ListaDubluInlantuita:

- Variabile :

- ListaDubluInlantuita.inceput- nodul de început al listei

- ListaDubluInlantuita.sfarsit- nodul de sfârșit al listei

- Metode:

- Fereastra.main(): Metoda principală care lansează aplicația și creează o instanță a clasei Fereastra.

- ListaDubluInlantuita.adaugaLaSfarsit(int valoare)- metoda pentru adăugarea unui element la sfârșitul listei

- **ListaDublulnlantuita.afiseazaNegative(int a, int b)**- metoda pentru afisarea elementelor negative dintr-un interval dat. Metoda parcurge lista si afiseaza fiecare element care se afla in intervalul dat si este negativ.

2. **Clasa Nod** (clasă internă):

- **Variable de instanță:**

-**ListaDublulnlantuita.Nod.valoare**- valoarea stocata in nod

-**ListaDublulnlantuita.Nod.urmator**- referinta catre urmatorul nod

din lista

-**ListaDublulnlantuita.Nod.anterior**- referinta catre nodul anterior din

lista

- **Constructor:**

- **Nod(int valoare)**: Constructorul clasei Nod.

3. **Clasa Fereastra** (clasă internă):

- **Variable de instanță:**

-**Fereastra.campElement**- campul de text pentru introducerea unui element

-**Fereastra.campA**- campul de text pentru introducerea limitei inferioare a intervalului

-**Fereastra.campB**- campul de text pentru introducerea limitei superioare a intervalului

-**Fereastra.zonaRezultat**- zona de text unde se afiseaza rezultatele

- **Metode:**

- **Fereastra()**: Constructorul clasei Fereastra.

- **initializeazaFereastra()**: Metodă initializeaza fereastra. Seteaza titlul ferestrei, dimensiunea ferestrei, layout-ul panoului principal si adauga componentele la panoul principal.

Implementare:

Implementarea clasei ListaDublulnlantuita

Clasa **ListaDublulnlantuita** implementeaza o lista dubla inlantuita. Are doua variabile de instanta, **inceput** si **sfarsit**, care reprezinta nodurile de inceput si de sfarsit ale listei. Are si o clasa interna, **Nod**, care reprezinta un nod din lista.

Clasa **Nod** are trei variabile de instanta, **valoare**, **urmator** si **anterior**, care reprezinta valoarea stocata in nod, referinta catre urmatorul nod din lista si referinta catre nodul anterior

din lista.

Metoda **adaugaLaSfarsit(int valoare)** adauga un element la sfarsitul listei. Daca lista este goala, elementul nou devine primul si ultimul nod din lista. Daca lista nu este goala, elementul nou devine urmatorul nod dupa nodul de sfarsit.

Iata implementarea metodei **adaugaLaSfarsit(int valoare)**:

```
public void adaugaLaSfarsit(int valoare) {  
    Nod nodNou = new Nod(valoare);  
    if (inceput == null) {  
        inceput = nodNou;  
        sfarsit = nodNou;  
    } else {  
        sfarsit.urmator = nodNou;  
        nodNou.anterior = sfarsit;  
        sfarsit = nodNou;  
    }  
}
```

-In primul rand, se creeaza un nou nod cu valoarea data ca parametru.

-In al doilea rand, daca lista este goala, elementul nou devine primul si ultimul nod din lista.

-In al treilea rand, daca lista nu este goala, elementul nou devine urmatorul nod dupa nodul de sfarsit.

Metoda **afiseazaNegative(int a, int b)** afiseaza elementele negative dintr-un interval dat. Metoda parcurge lista si afiseaza fiecare element care se afla in intervalul dat si este negativ.

Iata implementarea metodei **afiseazaNegative(int a, int b)**:

```
public String afiseazaNegative(int a, int b) {  
    Nod curent = inceput; // incepem parcurgerea listei de la primul nod  
    StringBuilder rezultat = new StringBuilder(); // folosim un StringBuilder pentru a construi rezultatul  
  
    rezultat.append("Numere negative in intervalul [").append(a).append(", ").append(b).append("]: ");  
    while (curent != null) { // parcurgem lista pana cand ajungem la sfarsit  
        // daca valoarea nodului curent este in intervalul [a, b] si este negativa  
        if (curent.valoare >= a && curent.valoare <= b && curent.valoare < 0) {  
            rezultat.append(curent.valoare).append(" "); // adaugam valoarea la rezultat  
        }  
        curent = curent.urmator; // trecem la urmatorul nod  
    }  
    return rezultat.toString(); // convertim rezultatul la String si il returnam  
}
```

Implementarea clasei Fereastră

Clasa **Fereastră** implementeaza o fereastră grafica. Are cinci variabile de instanta, **campElement**, **campA**, **campB**, **zonaRezultat** si **lista**.

- **campElement** si **campA** sunt campuri de text pentru introducerea unui element si a limitei inferioare a intervalului,

- **campB** este campul de text pentru introducerea limitei superioare a intervalului,

- **zonaRezultat** este zona de text unde se afiseaza rezultatele

- **lista** este o instanță a clasei **ListaDubluInlantuita**.

Metoda **initializeazaFereastra()** initializează fereastra. Setează titlul ferestrei, dimensiunea ferestrei, layout-ul panoului principal și adaugă componentele la panoul principal.

Iată implementarea metodei **initializeazaFereastra()**:

```
private void initializeazaFereastra() {
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setTitle("Lista Dublu Inlantuita");
    setSize(400, 300);

    JPanel panouPrincipal = new JPanel();
    panouPrincipal.setLayout(new GridLayout(6, 2));

    // cream etichetele si campurile de text
    JLabel etichetaElement = new JLabel("Element:");
    campElement = new JTextField();
    JLabel etichetaA = new JLabel("Interval [a]:");
    campA = new JTextField();
    JLabel etichetaB = new JLabel("Interval [b]:");
    campB = new JTextField();

    // cream butoanele si le adaugam actiuni
    JButton butonAdauga = new JButton("Adauga Element");
    butonAdauga.addActionListener(e -> {
        int element = Integer.parseInt(campElement.getText());
        lista.adaugaLaSfarsit(element);
        zonaRezultat.append("Elementul " + element + " a fost adaugat.\n");
    });
    JButton butonAfiseaza = new JButton("Afiseaza Negative");
    butonAfiseaza.addActionListener(e -> {
        int a = Integer.parseInt(campA.getText());
        int b = Integer.parseInt(campB.getText());
        String rezultat = lista.afiseazaNegative(a, b);
        zonaRezultat.append(rezultat + "\n");
    });
}
```

Exemple de utilizare:

Iată câteva exemple de utilizare a programului:

Adaugare element

Element: 10

Afisare elemente negative

Interval [a]: -5

Interval [b]: 10

Numere negative in intervalul [-5, 10]: -1 -2 -3 -4 -5

Adaugare mai multe elemente

Element: 10

Element: 20

Element: -9

Afisare elemente negative

Interval [a]: -9

Interval [b]: 20

Numere negative in intervalul [-9, 20]: -1 -2 -3 -4 -5 -6 -7 -8 -9

Testare:

Problema 39. Să se interclaseze 2 liste ordonate crescător (în prealabil) – interclasarea presupune construcția unei liste ordonate crescător, treptat, din cele 2 liste ordonate crescător, astfel: la un moment dat se compară 2 elemente, unul din prima listă și unul din a doua listă și cel mai mic dintre ele va fi “propagat” în lista rezultat, după care procesul se reia. Ex. I1: 1 3 5 20 40 I2: 2 4 5 15 25 I: 1 2 3 4 5 5 15 20 25 40

Proiectare:

Pentru rezolvarea problemei am implementat o structura de date de tip lista simplu inlantuita. Datele introduse de utilizator sunt procesate de catre metode cu scopul rezolvarii problemei. Datele sunt introduse de catre utilizator in doua obiecte de tip lista simplu inlantuita, sunt procesate de metode de sortare si inserate intr-o a treia lista in ordine crescatoare pentru a fi afisate drept rezultat.

DIAGRAMA DE CLASE: Placeholder

Implementare:

Clasa Link - pentru a reprezenta un nod in lista:

Variabile:

- dData: Variabila care stocheaza valoarea nodului.
- next: Referinta catre urmatorul nod din lista.

Metode:

- Link(double dd): Constructor care initializeaza un nod cu o valoare data.
`public Link(double dd) { dData = dd; }`
- displayLink(): Afiseaza valoarea nodului sub forma de sir de caractere.
`public void displayLink() { System.out.print "{" + dData + " "; }`

Clasa LinkedList - pentru a reprezenta lista

Variabile:

- first: Referinta catre primul nod din lista.

Metode:

- LinkedList(): Constructor care initializeaza lista ca fiind goala.
`public LinkedList() { first = null; }`
- isEmpty(): Verifica daca lista este goala.
`public boolean isEmpty() { return (first == null); }`
- insertFirst(double dd): Insereaza un nod la inceputul listei.
`public void insertFirst(double dd) { Link newLink = new Link(dd); newLink.next = first; first = newLink; }`
- displayList(): Afiseaza toate nodurile din lista.
`public void displayList() { System.out.print("List (first-->last): "); Link current = first; while (current != null) { current.displayLink(); current = current.next; } System.out.println(""); }`
- Sort(): Sorteaza lista folosind algoritmul bubble sort.

```
public void Sort() {  
    int swapped;  
    Link current;  
    Link last = null;  
  
    if (first == null) {  
        return;  
    }  
  
    do {  
        swapped = 0;  
        current = first;  
  
        while (current.next != last) {  
            if (current.dData > current.next.dData) {  
                double temp = current.dData;  
                current.dData = current.next.dData;  
                current.next.dData = temp;  
                swapped = 1;  
            }  
  
            current = current.next;  
        }  
    } while (swapped != 0);  
}
```

- Insert(double value): Insereaza un nod in ordine crescatoare.

```
private void Insert(double value) {  
  
    Link newLink = new Link(value);  
  
    if (isEmpty() || value <= first.dData) {  
        newLink.next = first;  
        first = newLink;  
    } else {  
        Link current = first;  
  
        while (current.next != null && current.next.dData < value) {  
            current = current.next;  
        }  
  
        newLink.next = current.next;  
        current.next = newLink;  
    }  
}
```

- Merge(LinkList list1, LinkList list2): Combina doua liste sortate intr-o a treia lista sortata.

```
Java ~ Copy Caption ...  
  
public void Merge(LinkList list1, LinkList list2) {  
  
    Link current1 = list1.first;  
    Link current2 = list2.first;  
  
    while (current1 != null || current2 != null) {  
        if (current1 == null) {  
            Insert(current2.dData);  
            current2 = current2.next;  
  
        } else if (current2 == null) {  
  
            Insert(current1.dData);  
            current1 = current1.next;  
  
        } else {  
  
            if (current1.dData < current2.dData) {  
                Insert(current1.dData);  
                current1 = current1.next;  
  
            } else {  
                Insert(current2.dData);  
                current2 = current2.next;  
  
            }  
        }  
    }  
}
```

Clasa Study - clasa principala

Metode:

- Read(LinkList list, Scanner scanner): Citeste valori de la utilizator si le adauga in lista data.

```
Java  Copy Caption ...  
  
public static void Read(LinkList list, Scanner scanner) {  
  
    System.out.println("Enter values (Add an empty value to stop):");  
  
    while (true) {  
        String input = scanner.nextLine();  
  
        if (input.isEmpty()) {  
            break;  
        }  
  
        try {  
            double doubleValue = Double.parseDouble(input);  
            list.insertFirst(doubleValue);  
        } catch (NumberFormatException e) {  
            System.out.println("Invalid input. Please enter a valid double value.");  
        }  
    }  
}
```

- main: creeaza trei liste, introduce date, le sorteaza si le combina.

Programul creeaza trei obiecte de tip LinkList: list1, list2 si sortedList. Se utilizeaza un obiect Scanner pentru citirea datelor de la utilizator. Pentru fiecare lista (list1 si list2): se citesc valori de la utilizator, se sorteaza lista folosind algoritmul bubble sort, se afiseaza lista sortata. Se combina cele doua liste sortate intr-o a treia lista (sortedList). Se afiseaza lista rezultata.

Testare:

Testarea a fost efectuată pentru diferite date de intrare, cu scopul de a reduce posibilitatea esuarii produsului la livrare. Programul accepta date de intrare de tip double, iar în cazul în care utilizatorul introduce orice alt tip de date, programul raspunde corespunzător cu o eroare ce îl anunță că tipul de date este greșit.

```
try {  
    double doubleValue = Double.parseDouble(input);  
    list.insertFirst(doubleValue);  
} catch (NumberFormatException e) {  
    System.out.println("Invalid input. Please enter a valid double value.");  
}
```

Problema 48. Se dă o coadă de forma: $l: a_{11} a_{12} a_{13} a_{14} .. a_{1n} \rightarrow a_{21} a_{22} a_{23} a_{24} .. a_{2n} \rightarrow ... \rightarrow a_{m1} a_{m2} a_{m3} .. a_{mn}$ /, cu m și n date. Se cere să se construiască matricea asociată care conține m linii și n coloane, pe fiecare linie fiind elementele din câmpurile informație ale uneia din celulele listei, astfel prima linie va conține $a_{11} a_{12} a_{13} a_{14} .. a_{1n}$, a doua $a_{21} a_{22} a_{23} a_{24} .. a_{2n}$ samd.

Clasa Nod - pentru a reprezenta un nod in coada:

Variabile:

- val: Variabila care stocheaza valoarea nodului.
- next: Referinta catre urmatorul nod din coada.

Metode:

- Nod(int val): Constructor care initializeaza un nod cu o valoare data.

Clasa Coadă - pentru a reprezenta coada:

Variabile:

- front: Referinta catre primul nod din coada.
- rear: Referinta catre ultimul nod din coada.

Metode:

- Coadă(): Constructor care initializeaza coada ca fiind goala.
- insereaza(int val): Insereaza un nod la sfârșitul cozii.
- elimina(): Elimina si returneaza nodul de la inceputul cozii.

Clasa InterfataGrafica - pentru a reprezenta interfata grafica a aplicatiei:

Variabile:

- frame: Cadru pentru interfata grafica.

- mTextField, nTextField, coadaTextField: Câmpuri de text pentru introducerea datelor de către utilizator.
- rezultatTextArea: Zona de text pentru afisarea rezultatelor.

Metode:

- InterfataGrafica(): Constructor care initializeaza interfata grafica.
- afiseazaFereastra(): Afiseaza fereastra interfetei grafice.
- extrageMatrice(String coada): Extrage o matrice dintr-un sir de caractere reprezentând o coada.
- afiseazaMatrice(Coada coada, int m, int n): Afiseaza o matrice construita dintr-o coada.
- proceseazaCoadaNumerica(): Proceseaza o coada numerica introdusa de utilizator, construiesc o matrice din ea si o afiseaza.

Clasa PROIECT_ISW - clasa principala:

Metode:

- main: Creeaza o instanta a clasei InterfataGrafica si afiseaza fereastra interfetei grafice.

Codul creeaza o structura de coada folosind o lista simplu inlantuita. Fiecare nod in coada are o valoare si o referinta la urmatorul nod. Coada are metode pentru a insera un nod la sfarsitul cozii si pentru a elimina nodul de la inceputul cozii. Interfata grafica permite utilizatorului sa introduca o coada numerica si dimensiunile unei matrici. Coada numerica este transformata intr-o matrice, care este apoi afisata in interfata grafica. In esenta, acest cod permite utilizatorului sa introduca o coada numerica, sa o transforme intr-o matrice si sa o afiseze.

Proiectare

Codul este proiectat în jurul conceptului de cozi și interfețe grafice. Clasa PROIECT_ISW este clasa principală a proiectului care conține metoda main. Aceasta creează și afișează o interfață grafică. Există, de asemenea, clase interne pentru un nod (Nod) și o coadă (Coadă), precum și o clasă pentru interfața grafică (InterfataGrafica).

Implementare

Implementarea fiecărei clase este destul de directă. Fiecare clasă are un constructor și metode pentru operațiuni de bază, cum ar fi inserarea și eliminarea elementelor din coadă. Clasa InterfataGrafica este responsabilă pentru crearea interfeței grafice și manipularea datelor introduse de utilizator.

De exemplu, metoda extrageMatrice extrage o coadă dintr-un șir de caractere introdus de utilizator, iar metoda afiseazaMatrice afișează o matrice în zona de text a interfeței grafice.

Testare

Introduceți număr de coloane și linii, apoi introducem coada 1 2 3 4 → 5 6 7 8 → 9 10 11 12 13, apasam butonul " Creeaza matricea".

Se dă o coadă de forma: $l: a_{11} a_{12} a_{13} a_{14} \dots a_{1n} \rightarrow a_{21} a_{22} a_{23} a_{24} \dots a_{2n} \rightarrow \dots \rightarrow a_{m1} a_{m2} a_{m3} \dots a_{mn}$, cu m și n date. Se cere să se construiască matricea asociată care conține m linii și n coloane, pe fiecare linie fiind elementele din cîmpurile informație ale uneia din celulele listei, astfel prima linie va conține $a_{11} a_{12} a_{13} a_{14} \dots a_{1n}$, a doua $a_{21} a_{22} a_{23} a_{24} \dots a_{2n}$ samd.

m= n=

Introduceți coada:

*ATENȚIE! Pentru delimitarea cozii folosiți →

```
1 2 3 4
5 6 7 8
9 10 11 12
```

6. Ghid Utilizator

La deschiderea aplicației, sunteți întâmpinați de meniul principal (fig. 1), ce conține text sugestiv și cinci butoane ce conțin numele fiecărui participant la dezvoltarea aplicației.



Fig. 1 - Meniul Principal

În urma accesării butonului marcat cu săgeata (1), (fig. 2), veți fi redirectionat către o fereastră nouă.

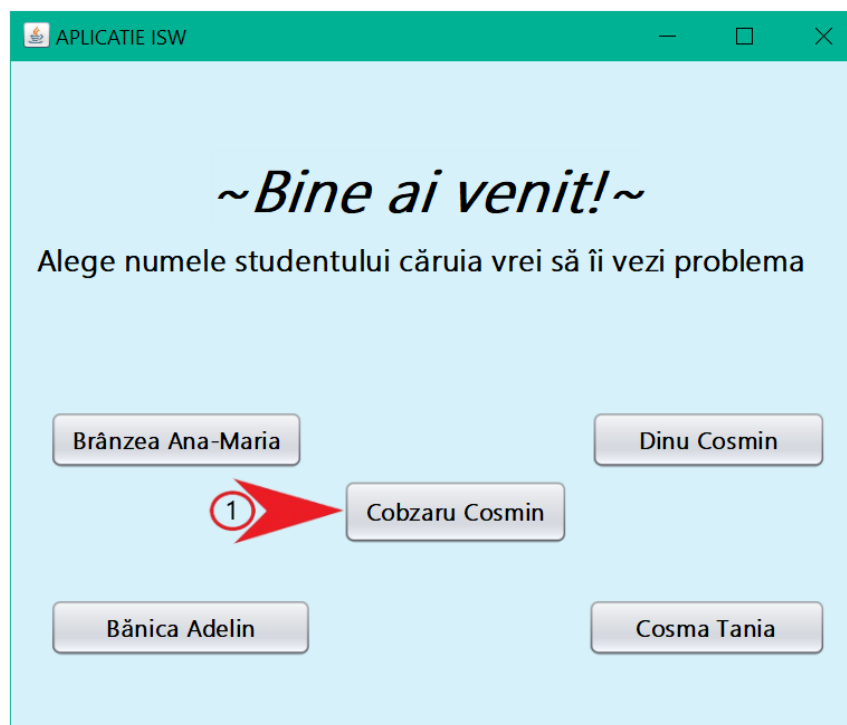


Fig. 2 - Accesarea unui buton

Noua fereastră (fig. 3) conține rezolvarea problemei atribuite participantului într-un format comun cu restul ferestrelor ce conțin rezolvări de probleme. Săgeata (1) indică cerința problemei. Săgețile (2) și (3) indică casute în care puteți introduce valori pentru rezolvarea problemei. Săgeata (4) reprezintă butonul de adăugare a valorilor în

aplicatie pentru rezolvarea problemei. Sageata (5) reprezinta butonul de stergere ale datelor introduse anterior. Sageata (6) genereaza rezultatul problemei in urma introducerii datelor. Sageata (7) reprezinta casuta in care se ofera rezultatul. Sageata (8) reprezinta butonul ce va redirectioneaza catre meniul principal pentru a selecta o alta problema.

Note: Restul de ferestre ce contin rezolvări de probleme urmează același format. Asadar, veți observa că enunțul problemei, casutele de adăugare de valori, butoanele de adăugare și stergere, casuta de rezultat și butonul de redirectionare la meniul principal sunt prezente și în acele ferestre.

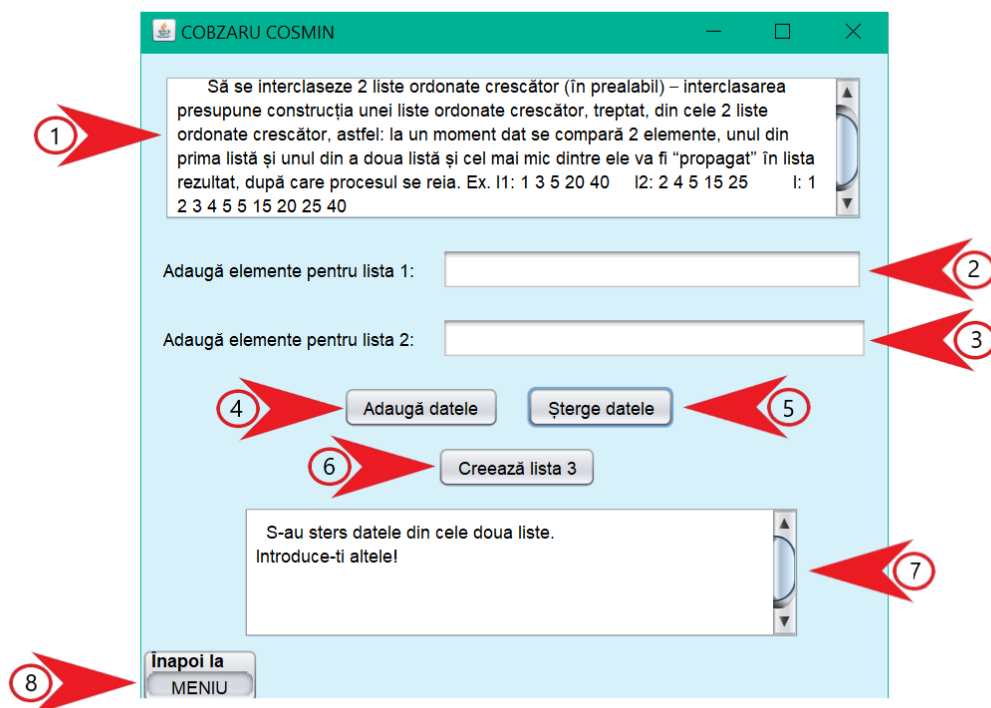


Fig. 3 - Fereastra ce contine rezolvarea unei probleme

Intorcandu-ne in meniul principal, putem accesa o alta problema (1) (fig. 4).

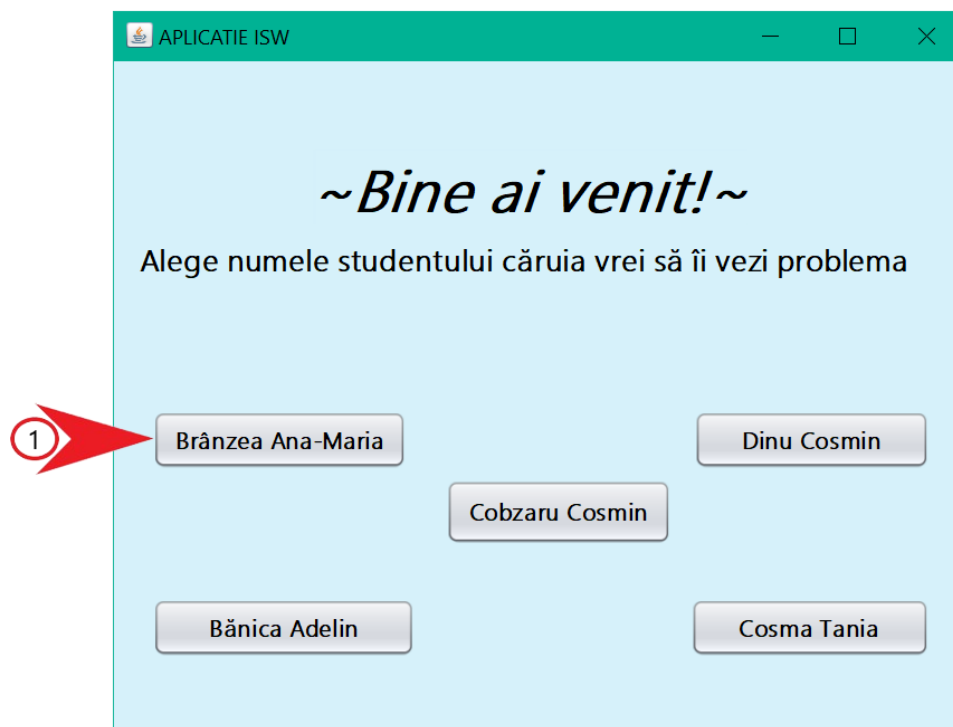


Fig. 4

Se pot observa similaritati între fereastra anterioară ce conține rezolvarea unei probleme (fig. 3) și fereastra ce conține rezolvarea altei probleme (fig. 5). Accesând butonul din (fig.5) obținem rezultatul problemei.

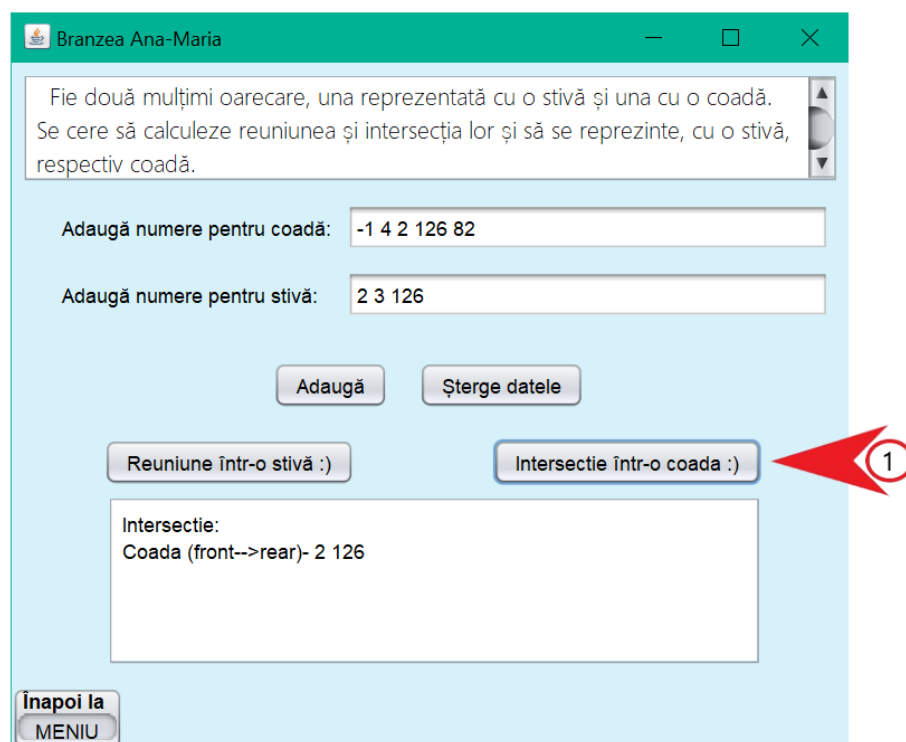


Fig. 5 - Fereastra ce conține rezolvarea unei probleme

Fereastra din (fig. 5) conține două butoane de rezolvare pentru problema respectivă. Prin urmare, accesând al doilea buton marcat de săgeata (1) din (fig. 6), obținem un al doilea rezultat pentru acea problema.

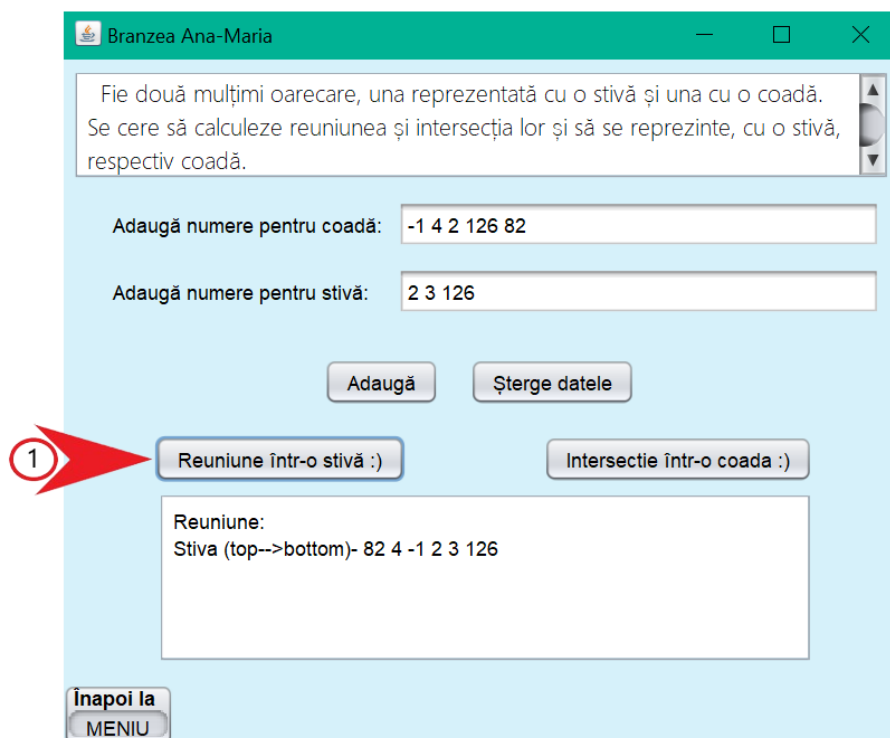


Fig. 6

Ne întoarcem în meniul principal și repetăm pașii pe care i-am efectuat în (fig. 4) pentru a naviga restul ferestrelor reprezentate de numele participanților (fig. 7), (fig 8)



Fig. 7

În (fig. 8) se poate observa o fereastră de rezolvare a unei probleme urmând același format precum restul ferestrelor de rezolvare. Casutele de text reprezentate de săgețile (1), (2) și (3) sunt zone de introducere ale datelor de intrare pentru rezolvarea problemei. Butonul indicat de săgeata (4) oferă rezultatul problemei.

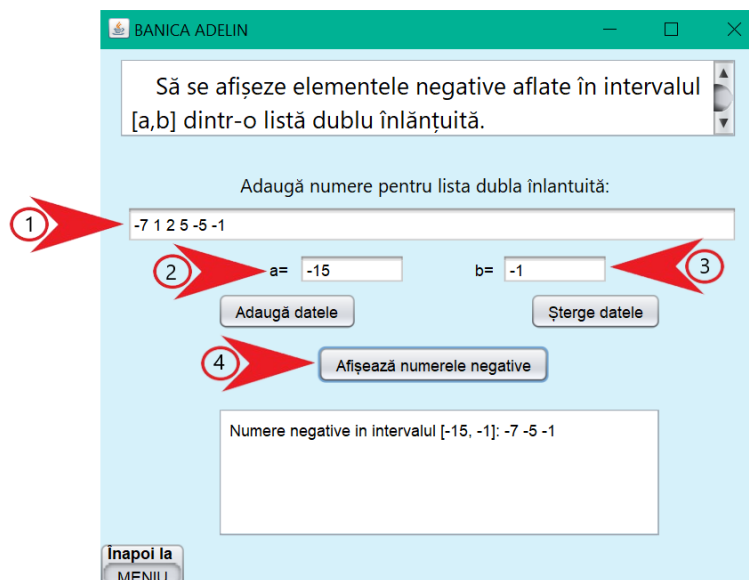


Fig. 8

Repetam pasii pentru a ne intoarce in meniul principal si navigam spre ferestrele de rezolvare ale ultimelor probleme (fig. 9) (fig. 10).



Fig. 9

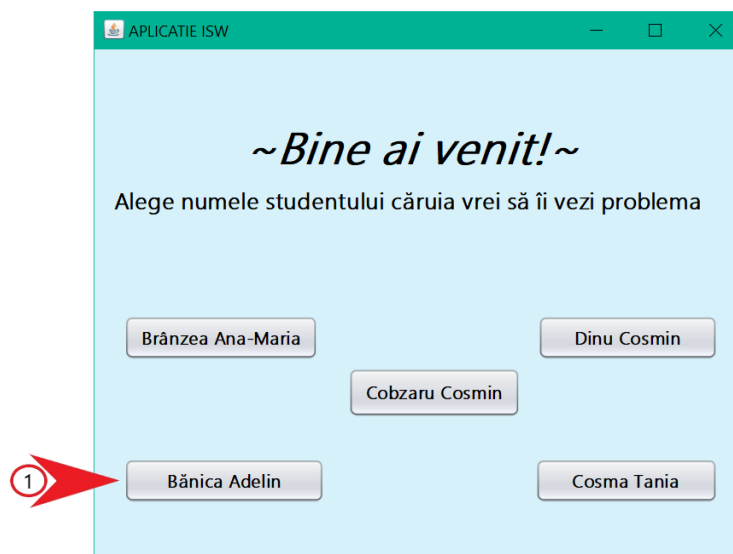


Fig. 10

În final avem rezolvările ultimelor două probleme în (fig.11) (fig. 12).

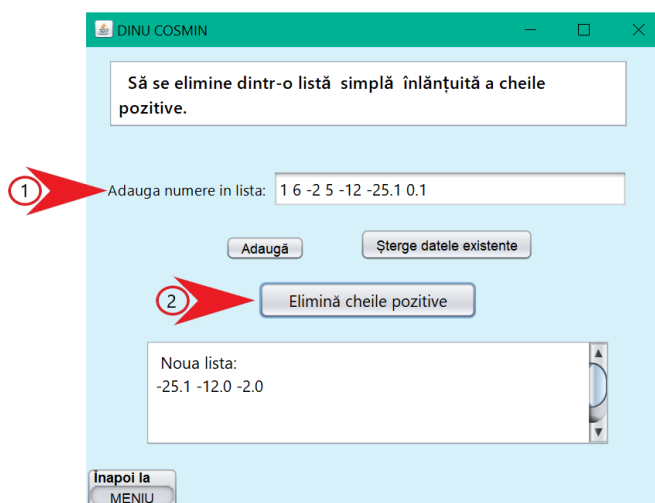


Fig. 11

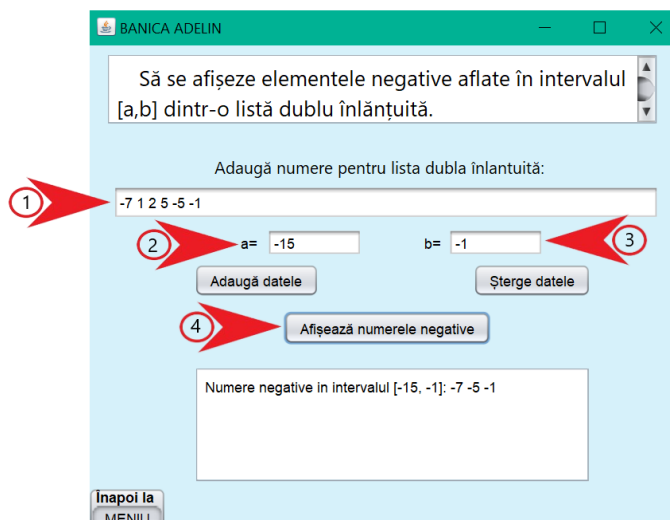


Fig. 12

În (fig. 12) se poate observa o fereastră de rezolvare a unei probleme urmând același format precum restul ferestrelor de rezolvare. Casutele de text reprezentate de săgețile (1), (2) și (3) sunt zone de introducere ale datelor de intrare pentru rezolvarea problemei. În casuta (1) adăugăm numere pentru lista dublu înlănțuită, în casuta (2) și (3) limitele intervalului folosind butonul “Adaugă date”, respectiv putem șterge date folosind butonul “Șterge datele”. Butonul indicat de săgeata (4) oferă rezultatul problemei, mai precis afișarea numerelor negative din intervalul ales.

7. Documentatie de sistem

Meniu Principal

- Utilizatorul lansează aplicația și este întâmpinat de un meniu principal intuitiv.
- Meniul principal oferă butoane pentru accesarea diferitelor funcționalități ale aplicației.
- Utilizatorul poate naviga în interiorul meniului principal și poate accesa ferestrele funcționale dorite.



Ferestre de Rezolvare

- După selectarea unei funcționalități, utilizatorul este direcționat către o fereastră de rezolvare specifică.
- Fiecare fereastră de rezolvare conține enunțurile problemelor, zone de text pentru introducerea valorilor, butoane pentru efectuarea operațiilor și câmpuri pentru afișarea rezultatelor.

The image displays four distinct windows from a software application, each designed for solving a specific problem. Each window features a title bar with the user's name, a descriptive text area for the problem, input fields for data, and buttons for performing operations.

- BIANZEA ANA-MARIA:** The problem involves two multisets, one represented by a stack and the other by a queue. The goal is to calculate their union and intersection. The interface includes input fields for adding numbers to the stack or queue, buttons for adding or deleting data, and buttons for performing union and intersection operations. A 'Back to Menu' button is also present.
- DINU COSMIN:** The problem is to remove all positive elements from a simple linked list. The interface includes an input field for adding numbers to the list, buttons for adding or deleting existing data, and a button to remove positive elements. A 'Back to Menu' button is also present.
- CORBARIU COSMIN:** The problem is to interleave two sorted lists (in pre-order) to create a new sorted list. The interface includes input fields for adding elements to two separate lists, buttons for adding or deleting data, a button to create the third list, and a 'Back to Menu' button.
- COSMA TANIA:** The problem is to construct a matrix based on given input. The interface includes input fields for 'm' and 'n', a text area for inputting the queue, a button to create the matrix, and a 'Back to Menu' button.

Detalii Tehnice

- Limbaj de Programare: Java
- Mediu de Dezvoltare Integrat (IDE): NetBeans
- Biblioteci Grafice: Swing pentru dezvoltarea interfeței grafice
- Metodologie de Dezvoltare: Modelul Waterfall pentru planificare și paradigma Agile pentru rezolvarea problemelor și proiectare.

Cerințe Minime Hardware

- Procesor: Dual-core, 2 GHz sau echivalent
- RAM: 4 GB
- Spațiu pe Disc: 10 GB liber
- Rezoluție Minimă a Ecranului: 1280x800

Librării și Module Utilizate

- Swing: Pentru dezvoltarea interfeței grafice.
- Java.util: Pentru gestionarea structurilor de date și a fluxurilor de date.
- Junit: Pentru testare unitară.

Împărțirea Aplicației în Module

Module pentru Meniul Principal

- MeniuFrame.java

Module pentru Ferestrele de Rezolvare

- AdelinFrame.java
- CosminCFrame.java
- CosminDFrame.java
- Tania.java
- Interfatapb53.java